Chapter 10
Hierarchical Data Format 5 : HDF5

Abstract In this chapter we introduce the Hierarchical Data Format 5 (HDF5) specification. We present a number of examples which use the HDF5 API for reading and writing HDF files. We present C/C++ API examples of the API including examples of writing, reading compound types in HDF5. For large scientific data, HDF5 is representative of the APIs which are used in CDF and other data formats containing simulation or experimentally collected data.

Contents

10.1 HDF5 files 191
10.2 Example of HDF5 API 193

Scientific datasets can be enormously large in size (comprising of many terabytes of data). It is thus essential that large datasets are transported and accessed in an efficient and standardized manner (otherwise multiple tools will have to reinvent the wheel of doing performance optimized IO on large datasets). Since scientific datasets also have a lot of structure to them, efficient storage policies can be adopted.

HDF5 is a hierarchical data format specification and supporting library implementation. HDF version 5 addresses some of the limitations of the older HDF formats, such as (i) 2 Gb file size restriction, (ii) cap on maximum number of objects per file, and (iii) library source of the previous HDF was old and outdated. HDF5 also includes the following improvements to the previous version: (i) new file format (which removes the restrictions on file size), (ii) simpler, more comprehensive data model (which incidentally has only two basic structures: a multi-dimensional array of record structure and a grouping structure, and (iii) simpler library with improved support of parallel I/O and threading.

10.1 HDF5 files

HDF5 are organized in a hierarchical structure around two basic objects: (i) groups and (ii) datasets. *Datasets* contain the actual multi-dimensional data elements (along-

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_10, © Springer Science+Business Media, LLC 2011

with supporting metadata), while *groups* provide the data with organizational skeleton; a group contains instances of other groups or datasets (alongwith supporting metadata). The *metadata* is actually data about the data. In addition to metadata, groups and datasets may have an associative *attribute* list. Attributes are user-defined HDF5 structures containing extra information about the HDF5 object.

HDF5 groups and group members are deliberately organized in a manner analogous to the UNIX file system hierarchy, with `/` representing the *root* group, and `/foo` denoting a group `foo` which is a member of the root group. An HDF5 group comprises of: (i) group header (consisting of group name and list of attributes), and (ii) group symbol table (which is a list of the HDF5 objects which belong to the group).

An HDF5 dataset comprises of: (i) dataset header, and (ii) data array. The header contains information needed to interpret the array; it contains the name of the object (header name), its dimensionality (header dataspace), number-type (header datatype), and disk storage information (header storage layout). The name of the object is an ASCII string.

The dimensionality of the dataset is stored in the dataspace in the header. The dimensions can be static or *unlimited* (extensible). Properties of the dataspace consist of its *rank* (number of dimensions) of the data array, actual sizes of the dimensions, and the maximum size of the dimensions. Importantly, a dataspace can also describe portions of the dataset, allowing input-output operations on a *hyperslab* of the dataset. Selection of such a region is supported in HDF5 using the dataspace interface (H5S). The region(s) can be (i) contiguous, (ii) non-contiguous hyperslab, (iii) union of hyperslabs, and (iv) list of independent points.

The HDF5 supported data-types are (i) integer, (ii) floating point (IEEE 32-bit and 64-bit) in both endian formats, (iii) references and (iv) strings. Symbolically the data-types can be referenced using the API enumerations, e.g., for **`int`** the corresponding HDF5 data-type is `H5T_NATIVE_INT`, similarly for **`long double`**, `H5T_NATIVE_LDOUBLE`. A C language **`struct`** can be represented as an HDF5 *compound* data-type.

The *storage* of the HDF5 data on disk is possible in a number of ways, with contiguous storage being the default. In this default mode, there is a one-to-one correspondence between data items in memory and disk. The other types of storage are: (ii) compact (useful when data is small and can be stored alongwith the header), and (iii) chunked (data is divided into equal sized chunks, which are stored separately). Chunked data increases performance, especially when accessing subsets of datasets. The HDF5 appears to the user as a directed graph whose nodes are HDF5 objects such as: (i) groups, (ii) datasets, (iii) datatypes and (iv) dataspaces. At the lowest level, an HDF5 file is comprised of:

1. A super block,
2. B-tree nodes: containing either symbol, nodes or raw data chunks,
3. Object headers:
4. Local heaps:
5. Free space.

Although the APIs provide opaque access to the underlying data, it is useful to understand how the underlying data is stored on disk file for improving efficiency. As an example, a HDF5 *group* is an object header in the HDF5 file. This object header contains a message that points to a local heap and to a B-tree node pointing to symbol nodes.

10.1.1 HDF5 API Naming Conventions

The naming convention of the HDF5 API are given below:

1. H5F: file-level access, e.g., `H5Fopen`,
2. H5G: group functions, e.g., `H5Gset`,
3. H5T: datatype functions, e.g., `H5Tcopy`,
4. H5S: dataspace functions, e.g., `H5Screate_simple`,
5. H5D: dataset functions, e.g., `H5Dread`,
6. H5P: property list functions, e.g., `H5Pset_chunk`,
7. H5A: attribute access, e.g., `H5Aget_name`,
8. H5Z: compression registration, e.g., `H5Zregister`,
9. H5E: error handling routine, e.g., `H5Eprint`,
10. H5R: reference function, e.g., `H5Rcreate`,
11. H5I: identifier routing, e.g., `H5Iget_type`.

10.2 Example of HDF5 API

We present two simple examples of using the HDF5 API. In Listing 10.1 we have presented a HDF5 dataset writer, which writes a two-dimensional table of 32-bit integers to a disk file.

```
// \file hdf_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using HDF5 for data storage
#include <cstdio>              // for C FILE
#include <cassert>             // assertion checking
#include <cstdlib>             // exit
#include <iostream>            // program IO
#include <hdf5.h>              // HDF5

static const char fileName[] = "example_A.h5";
static const char datasetName[] = "ASTR_10";
static const unsigned int DIM0 = 3;
static const unsigned int DIM1 = 3;

static void InitializeDataSet( int  D[DIM0][DIM1] ) {
  for( unsigned int i=0; i < DIM0; ++i )
    for( unsigned int j=0; j < DIM1; ++j )
      D[i][j] = i+j;
}

int main( int argc, char *argv[] ) {
```

```
  hid_t file, space, dataset;          // Opaque HDF5 handles
  herr_t status;
  hsize_t dims[2] = { DIM0, DIM1 };
  int     write_data[DIM0][DIM1];           // write buffer
  InitializeDataSet( write_data );
  file = H5Fcreate( fileName, H5F_ACC_TRUNC, H5P_DEFAULT, H5P_DEFAULT );
  space= H5Screate_simple( 2, dims, NULL );
  dataset = H5Dcreate( file, datasetName, H5T_STD_I32LE, space,
            H5P_DEFAULT, H5P_DEFAULT, H5P_DEFAULT );
  status = H5Dwrite( dataset, H5T_NATIVE_INT, H5S_ALL, H5S_ALL,
         H5P_DEFAULT, write_data[0] );
  status = H5Dclose( dataset ); // first close the dataset
  status = H5Sclose( space );   // then close the space
  status = H5Fclose( file );    // and lastly close the file.
  return (0);
}
```

Listing 10.1 Example of using HDF5 library API for writing dataset

The corresponding *reader* example is presented in Listing 10.2. The same file is opened using `H5F_ACC_RDONLY` mode, and the contents are verified and printed to the screen.

```
// \file hdf_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using HDF5 for data storage
#include <cstdio>              // for C FILE
#include <cassert>             // assertion checking
#include <cstdlib>             // exit
#include <iostream>            // program IO
#include <hdf5.h>              // HDF5

static const char fileName[] = "example_A.h5";
static const char datasetName[] = "ASTR_10";
static const unsigned int DIM0 = 3;
static const unsigned int DIM1 = 3;

static void CheckAndPrintDataSet( int  D[DIM0][DIM1] ) {
  for( unsigned int i=0; i < DIM0; ++i ) {
    std::cout << std::endl;
    for( unsigned int j=0; j < DIM1; ++j ) {
      assert( D[i][j] == (int)(i+j) );
      std::cout << " " << D[i][j];
    }
  }
}

int main( int argc, char *argv[] ) {
  hid_t file, dataset;                      // Opaque HDF5 handles
  herr_t status;
  int     read_data[DIM0][DIM1];            // read buffer
  file = H5Fopen( fileName, H5F_ACC_RDONLY, H5P_DEFAULT );
  dataset = H5Dopen( file, datasetName, H5P_DEFAULT );
  status = H5Dread( dataset, H5T_NATIVE_INT, H5S_ALL, H5S_ALL,
         H5P_DEFAULT, read_data[0] );
  // check and print dataset
  CheckAndPrintDataSet( read_data );
  status = H5Dclose( dataset ); // first close the dataset
  status = H5Fclose( file );    // and lastly close the file.
  std::cout << std::endl;
  return (0);
}
```

Listing 10.2 Example of using HDF5 library API for reading dataset

The SCons file for compiling these applications is shown for reference:

```
Program('hdf_example', ['hdf_example.cpp'],
        CXXFLAGS="-Wall -O0 -ggdb",
        LIBS=['hdf5','m','pthread'])
Program('hdf_read_example', ['hdf_read_example.cpp'],
        CXXFLAGS="-Wall -O0 -ggdb",
        LIBS=['hdf5','m','pthread'])
```

Compiling and executing the writer/reader programs gives:

```
$ ./hdf_example
$ ls -l example_A.h5
-rw-rw-r--. 1 skoranne example_A.h5
$ ./hdf_read_example

 0 1 2
 1 2 3
 2 3 4
```

In the above Listing 10.1, the use of HDF5 API functions is demonstrated. In HDF5, *datatype* and *dimensionality* are independent objects, which are created independently from any particular dataset they may be attached to. A dataspace can be created once the *rank* and *dimension* of the space are known; in the above example we use the `H5Screate_simple` function to create the dataspace.

The datatype is created using the `H5Tcopy(H5T_NATIVE_INT)` function (in the example it is simple integer), which creates a datatype using the native integer type as a template. The endian behavior of the datatype can be set using the `H5Tset_order` function; we can choose little-endian (`H5T_ORDER_LE`) or big-endian. Once the dataspace and datatype handles have been created the actual dataset can be created. At this point in time, the dataset is connected to the underlying data file, the datatype and the dataspace. The H5D function for creating a dataset is: `H5Dcreate`. In addition to the filename, the name of the dataset has to be given. Once the dataset has been created, actual data can be read or written to the disk file using the `H5Dread` and `H5Dwrite` functions. These functions take the pointer to the data for reading and writing.

The datatype, dataspace and dataset object handles should be released once they are no longer needed. Each handle must be released separately using: `H5Tclose` (for datatype), `H5Dclose` (for dataset), and `H5Sclose` (for dataspace).

Using the public API functions of the HDF5 library we can write a HDF inspection tool as shown in Listing 10.3.

```
// \file hdf_info.cpp
// \author Sandeep Koranne (C) 2010
// \description HDF5 info program
#include <cstdio>              // for C FILE
#include <cstring>             // for string function
#include <cassert>             // assertion checking
#include <cstdlib>             // exit
#include <iostream>            // program IO
#include <hdf5.h>              // HDF5

static const unsigned int MAX_NUMBER = 1024;
```

```
static char fileName[ MAX_NUMBER ];
static char datasetName[ MAX_NUMBER ];

int main( int argc, char *argv[] ) {
  if( argc != 3 ) {
    std::cerr << "Usage: hdf_info <h5> <dataset>..\n";
    exit(1);
  }
  strcpy( fileName, argv[1] );
  strcpy( datasetName, argv[2] );
  hid_t file, dataset;                      // Opaque HDF5 handles
  file = H5Fopen( fileName, H5F_ACC_RDONLY, H5P_DEFAULT );
  // analyze the file object
  hsize_t filesize;
  (void) H5Fget_filesize( file, &filesize );

  hsize_t objc[ 5 ];
  for( int i=0; i < 5; ++i ) objc[i] = 0;
  objc[0] = H5Fget_obj_count( file, H5F_OBJ_FILE ); // no. file objects
  objc[1] = H5Fget_obj_count( file, H5F_OBJ_DATASET ); // no. datasets
  objc[2] = H5Fget_obj_count( file, H5F_OBJ_GROUP ); // no. group objects
  objc[3] = H5Fget_obj_count( file, H5F_OBJ_DATATYPE ); // no. datatype
  objc[4] = H5Fget_obj_count( file, H5F_OBJ_ATTR ); // no. attributes
  std::cout << "File " << fileName
            << " File size = " << filesize
            << " free space = " << H5Fget_freespace( file ) << "\n"
            << " has " << objc[0] << " file objects\n"
            << " has " << objc[1] << " dataset objects\n"
            << " has " << objc[2] << " group objects\n"
            << " has " << objc[3] << " datatype objects\n"
            << " has " << objc[4] << " attribute objects\n";
  // in case we dont know the dataset name we can query it
  size_t max_dataset_number = objc[1];
  hid_t *dataset_objects = new hid_t[ max_dataset_number ];
  (void) H5Fget_obj_ids( file, H5F_OBJ_DATASET,
                         max_dataset_number, dataset_objects );

  for( size_t i=0; i < max_dataset_number; ++i ) {
    char temp_dataset_name[ MAX_NUMBER ];
    H5Fget_name( dataset_objects[i], temp_dataset_name, MAX_NUMBER );
    std::cout << "Dataset name = " << temp_dataset_name;
  }
  std::cout << std::endl;
  dataset = H5Dopen( file, datasetName, H5P_DEFAULT );
  hsize_t storage_size = H5Dget_storage_size( dataset );
  haddr_t offset = H5Dget_offset( dataset );
  std::cout << "\nStorage size : " << storage_size
            << "\nOffset       : " << offset
            << std::endl;

  hid_t datatype = H5Dget_type( dataset );
  hid_t hdf5class= H5Tget_class( datatype );
  switch( hdf5class ) {
  case H5T_INTEGER:{std::cout << "Data set class = INTEGER\n"; break;}
  case H5T_FLOAT:{std::cout << "Data set class = FLOAT\n"; break;}
  case H5T_TIME:{std::cout << "Data set class = TIME\n"; break;}
  case H5T_STRING:{std::cout << "Data set class = STRING\n";break;}
  case H5T_BITFIELD:{std::cout << "Data set class = BITFIELD\n";break;}
  case H5T_OPAQUE:{std::cout << "Data set class = OPAQUE\n";break;}
  case H5T_COMPOUND:{std::cout << "Data set class = COMPOUND\n";break;}
  case H5T_REFERENCE:{std::cout << "Data set class = REFERENCE\n";break;}
  case H5T_ENUM:{std::cout << "Data set class = ENUM\n";break;}
  case H5T_VLEN:{std::cout << "Data set class = VLEN\n";break;}
  case H5T_ARRAY:{std::cout << "Data set class = ARRAY\n";break;}
  case H5T_NO_CLASS:
  default:
    { std::cout << "Data set class = NO CLASS\n";  break; }
```

```
  }
  if( hdf5class == H5T_FLOAT ) {
    // check point normalization
    H5T_norm_t norm = H5Tget_norm( datatype );
    if( norm == H5T_NORM_NONE )
      std::cout << "Floating point is not normalized.\n";
    else
      std::cout << "Floating point is normalized.\n";
  } else if( hdf5class == H5T_INTEGER ) {
    // check integer signed type
    H5T_sign_t sign_t = H5Tget_sign( datatype );
    if( sign_t == H5T_SGN_2 )
      std::cout << "Integer is 2's complement.\n";
    else if( sign_t == H5T_SGN_NONE )
      std::cout << "Integer is UNSIGED.\n";
  }
  hid_t order = H5Tget_order( datatype );
  switch( order ) {
  case H5T_ORDER_LE:{std::cout << "Data order is Little Endian\n"; break;}
  case H5T_ORDER_BE:{std::cout << "Data order is Big Endian\n"; break;}
  case H5T_ORDER_VAX:{std::cout << "Data order is VAX\n"; break;}
  case H5T_ORDER_NONE:{std::cout << "Data order is NONE\n"; break;}
  }

  hid_t size = H5Tget_size( datatype );
  std::cout << "Data size = " << size << "\n";

  hid_t dataspace = H5Dget_space( dataset );
  hid_t dataspace_class = H5Sget_simple_extent_type( dataspace );
  switch( dataspace_class ) {
  case H5S_NO_CLASS:{std::cout<<"Dataspace class = H5S_NO_CLASS\n";break;}
  case H5S_SCALAR:{std::cout << "Dataspace class = H5S_SCALAR\n"; break;}
  case H5S_SIMPLE: {std::cout << "Dataspace class = H5S_SIMPLE\n";break;}
  case H5S_NULL: {std::cout << "Dataspace class = H5S_NULL\n"; break;}
  }
  hssize_t num_points = H5Sget_simple_extent_npoints( dataspace );
  hid_t rank = H5Sget_simple_extent_ndims( dataspace );
  hsize_t *dims = new hsize_t[ rank ];
  hsize_t *maxdims = new hsize_t[ rank ];
  for( int i=0; i < rank; ++i ) maxdims[i] = 32;
  (void)H5Sget_simple_extent_dims( dataspace, dims, maxdims );
  std::cout << "Num points = " << num_points << "\n";
  std::cout << "Is Simple ? = " << H5Sis_simple( dataspace ) << "\n";
  std::cout << "Rank = " << rank << "\n";
  for( int i=0; i < rank; ++i ) {
    std::cout << "dim[" << i << "] = " << dims[i] << "\n";
  }
  std::cout << std::endl;

  delete[] dims;
  H5Sclose( dataspace );
  H5Tclose( datatype );
  H5Fclose( file );
  H5Dclose( dataset );
  std::cout << std::endl;
  return (0);
}
```

Listing 10.3 HDF5 API example

When using the C++ API the following libraries should be linked with the application program:

```
Program('hdfcxx', ['hdfcxx.cpp'],
        CXXFLAGS="-Wall -O0 -ggdb",
        LIBS=['hdf5','hdf5_hl','hdf5_cpp',
```

```
              'hdf5_hl_cpp','pthread'])
```

Compiling and running this 'info' program on an HDF5 file gives:

```
./hdf_info example_A.h5 ASTR_10
File example_A.h5 File size = 2084 free space = 0
 has 1 file objects
 has 0 dataset objects
 has 0 group objects
 has 0 datatype objects
 has 0 attribute objects

Storage size : 36
Offset       : 2048
Data set class = INTEGER
Integer is 2's complement.
Data order is Little Endian
Data size = 4
Dataspace class = H5S_SIMPLE
Num points = 9
Is Simple ? = 1
Rank = 2
dim[0] = 3
dim[1] = 3
```

An example of using C++ API is shown in Listing 10.4.

```
// \file hdfcxx.cpp
// \author Sandeep Koranne (C) 2010
// \description HDF API in C++
#include <cstdlib>
#include <H5Cpp.h>
#include <iostream>

using namespace H5;

int main( int argc, char *argv[] ) {
  H5File file( argv[1], H5F_ACC_RDONLY );
  hid_t dataset_id = H5Dopen( file.getId(), argv[2], H5P_DEFAULT );
  if( dataset_id == -1 ) {
    std::cerr << "Dataset : " << argv[2] << " not found in file.\n";
    exit(1);
  }
  DataSet dataset( dataset_id );
  std::cout << "Storage size = " << dataset.getStorageSize();

  dataset.close();
  file.close();
  std::cout << std::endl;
  return (0);
}
```

Listing 10.4 HDF5 C++ API example

The primary use of HDF5 is in maintaining large datasets, for example, those coming from simulations of physical processes, or astronomy. In these cases the size of the dataset exceeds the installed physical memory on the computer system by orders of magnitude. For example, an 8 Gb machine may be expected to analyze a terabyte size file containing temperature and pressure readings from a simulation. HDF5 includes the ability to load into memory *sections* of data from disk.

The sections are arranged into *hyperslabs* which is a dimensional generalization of multi-dimensional arrays. See Figure 10.1 for a depiction of how data organized on disk can be loaded into memory. The data can be loaded from a contiguous slab, or accessed using *strides*. The function `H5Sselect_hyperslab` can be used to select regions from the dataspace, both in dataset space, as well as the memory space.

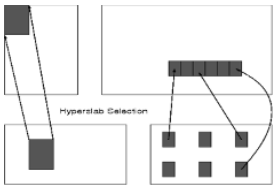


Fig. 10.1 Hyperslab data reading in HDF5

10.2.1 Writing and Reading compound datatype in HDF5

Writing and reading compound datatype (such as a C language structure) can be performed using HDF5 *groups*. Consider the example of temperature and pressure (as floating point values) readings taken at various coordinates (X,Y pairs). This organization can be written in C as:

```
typedef struct {
  int X,Y;
} Coord;
typedef struct {
  float temperature, pressure;
  Coord location;
  hobj_ref_t group;
} Reading;
```

The HDF5 API calls needed to implement writing and reading this compound type can be written as:

```
hid_t file = H5Fcreate( FILE, H5F_ACC_TRUNC, H5P_DEFAULT, H5P_DEFAULT );
space = H5Screate_simple( 2, dims, NULL );
dataset = H5Dcreate( file, ``LOC_READING'', H5T_NATIVE_DOUBLE,..);
group = H5Gcreate(file, ``SensorA'', H5P_DEFAULT, H5P_DEFAULT, H5P_DEFAULT);
(void)H5Gclose( group );
group = H5Gcreate(file, ``SensorB'',H5P_DEFAULT,H5P_DEFAULT,H5P_DEFAULT);
(void)H5Gclose( group );
Reading *readings = new Reading[4];
reading[0]->temperature = 100.0; reading[0]->pressure = 65.1;
reading[0]->X = 6, reading[0]->Y = 8;
reading[1]->temperature = 92.4; reading[1]->pressure = 47.2;
reading[1]->X = 12, reading[0]->Y = 18;
status = H5Rcreate( &reading[0].group, file, ``SensorA'', H5R_OBJECT, -1);
```

The *group* handle is stored in the data structure representing the location based readings. A group can be created using the `H5Gcreate` function. A hierarchical group can be created by providing the absolute name of the group to this function, e.g.:

```
group = H5Gcreate( file, ``/Data/Another_Group'');
```

creates a group "Another_Group" inside the group "Data" which is in the root group. A dataset can be placed in a group by giving the name of the group as the argument to the `HD5create` function. To open the dataset in a group, two methods can be used: (i) use the absolute name of the group in the `HD5open` function, or (ii) open the group, and pass the group handle to the `HD5open` function.

10.2.2 HDF5 Attributes

HDF5 attributes are small named datasets that are attached to primary datasets, or named datatypes. An attribute has two components: (i) name, and (ii) value. The value part contains one or more data entries of the same datatype. As mentioned above, H5A is the API prefix for all attribute related HDF5 functions. All the attributes of an object can be iterated using the *index value* identification of attributes. Since attributes are expected to be small, they are stored in the header of the object they are attached to. Attributes can be created using `H5Acreate`, an example is shown below:

```
id = H5Screate( H5S_SCALAR );
attr = H5Acreate( dataset, ``Integer'', H5T_NATIVE_INT, id, H5P_DEFAULT);
..
H5Aopen_name( dataset, ``Integer'' ); // reads same attribute back
```

10.2.3 References to objects

In HDF5 object references are based on the relative file address of the object header in the file and is deemed constant for the life of the object.

10.2.4 Conclusion

HDF5 is a versatile and efficient mechanism of data storage and access. Since it is widely supported on a number of high-performance computing platforms, availability of tools to analyze HDF5 files, and the use of multi-threaded I/O for reading and writing large datasets in HDF5 format can be expected. In the above section we have given a brief overview of the capabilities of HDF5.